Fiber Offers

System architecture, protocol schema, APIs, security model, and delivery stack.

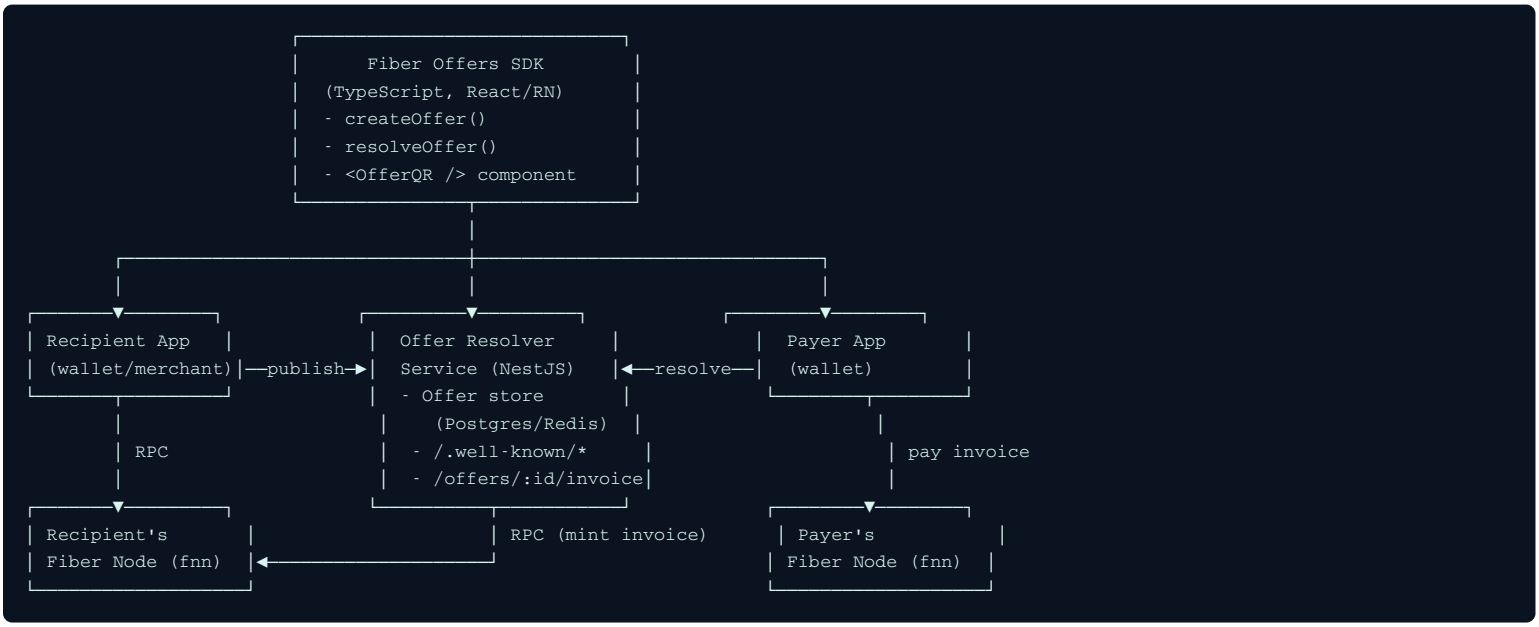
Document	**Technical Requirements Document (TRD)**
Category	**Wallet & Payment UX Infrastructure**
Author	**Leo — Dilamme / Independent**
Version	**1.0**
Status	**Draft for Hackathon Submission**

Table of Contents

1. System Overview

Fiber Offers adds two new components on top of an unmodified Fiber node (`fnn`): an **Offer Resolver Service** and a **client SDK**. Neither requires changes to Fiber's consensus, channel, or settlement logic — the system only produces standard Fiber invoices under the hood, which existing wallets already know how to pay.

1.1 Component diagram



1.2 Design principle

> **Non-invasive by design:** the Fiber node itself is never modified. All new logic lives in an application-layer service and SDK that talk to the node exclusively through its existing RPC surface. This keeps the project deployable against any standard Fiber node today, and keeps the migration path to a fully native (peer-message-based) transport open without a rewrite.

2. Core Components

Component	Responsibility	Language / Stack
Offer Schema & Codec	Canonical data model, signing, bech32m-style string encoding/decoding	TypeScript (shared package), Rust (optional native codec)
Offer Resolver Service	Stores published Offers, serves resolution requests, mints invoices via node RPC, exposes Fiber Address well-known endpoint	NestJS (Node.js/TypeScript), PostgreSQL, Redis/BullMQ
Recurrence Scheduler	Triggers resolution + payment attempts on a cadence, enforces spending caps	BullMQ workers on Redis (pattern reused from existing auth infrastructure)
Client SDK	createOffer, resolveOffer, payOffer helpers; QR/copy-link UI components	TypeScript, React, React Native
Demo Apps	Reference merchant app + reference payer wallet used for the hackathon demo	Next.js (merchant), Expo/React Native (payer)

3. Offer Data Model (Schema v1)

3.1 Canonical fields

Field	Type	Required	Description
`version`	uint8	Yes	Schema version, starts at `1`
`offer_id`	bytes32	Yes	Deterministic identifier, hash of the canonical payload
`node_id`	pubkey (33 bytes)	Yes	Recipient's Fiber node public key
`resolver_url`	string (optional)	No	HTTPS endpoint for MVP transport; omitted for native-transport-only Offers
`description`	UTF-8 string	No	Human-readable label ("Coffee Shop Checkout")
`assets`	array of AssetSpec	Yes	One or more accepted assets (CKB, UDT type script hash, or RGB++ asset id)
`amount_min` / `amount_max`	uint64 (optional)	No	Bounds on payable amount; both omitted = payer-specified ("tip jar") amount
`recurrence`	RecurrenceSpec (optional)	No	Interval, count/until, and per-cycle amount for subscription-style Offers
`expiry`	uint64 (optional, unix ts)	No	Optional absolute expiry of the Offer itself (not of individual invoices)
`single_use`	bool	Yes	Defaults to `false`; escape hatch to mark a specific Offer as one-time only
`signature`	bytes (Schnorr/ECDSA per Fiber's existing scheme)	Yes	Signs the canonical serialization of all preceding fields with the node's key

3.2 AssetSpec sub-object

Field	Type	Description
`asset_type`	enum	`ckb` | `udt` | `rgbpp`
`type_script_hash`	bytes32 (optional)	Required for `udt` / `rgbpp`
`symbol`	string	Display symbol, e.g. `USDI`

3.3 RecurrenceSpec sub-object

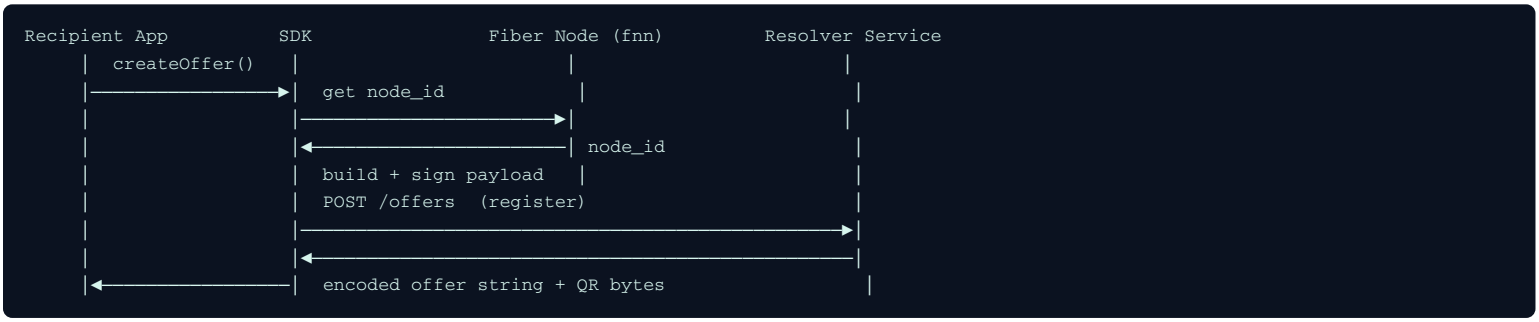
Field	Type	Description
`interval`	enum	`daily` | `weekly` | `monthly` | `custom_seconds`
`amount`	uint64	Fixed amount charged per cycle
`cap_cycles`	uint32 (optional)	Max number of charges; omitted = until revoked
`spending_cap_total`	uint64 (optional)	Hard ceiling enforced client-side; mandatory for any auto-pay implementation

3.4 Encoding

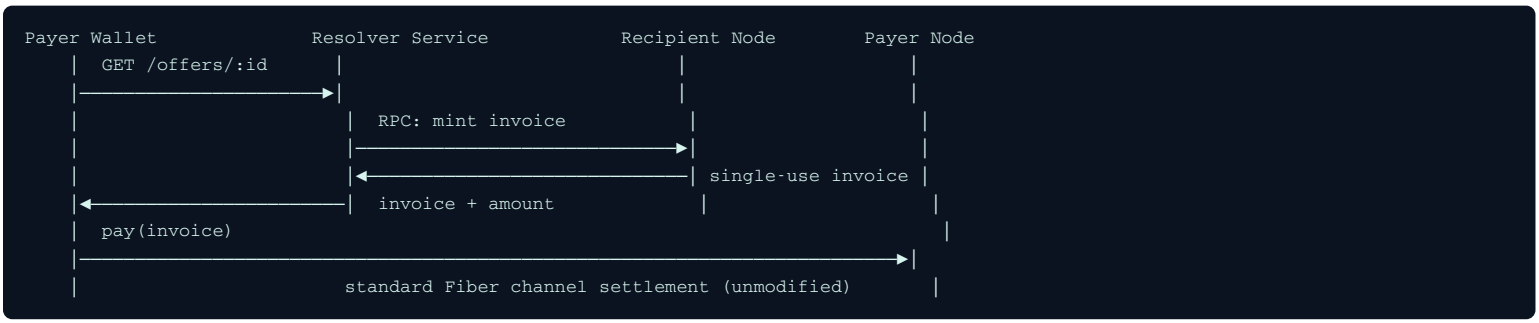
The Offer is serialized deterministically (protobuf-style canonical field ordering), then encoded as a bech32m string with an `fbroffer1` human-readable prefix, matching Fiber's existing address/invoice encoding conventions so wallets can reuse decode infrastructure with minimal changes. A companion JSON representation is used internally by the resolver service and SDK for readability and debugging.

4. Protocol Flows

4.1 Offer creation



4.2 Offer resolution & payment



4.3 Recurring cycle

1. Scheduler wakes on the Offer's recurrence interval.
2. Checks `spending_cap_total` and `cap_cycles` against amount already paid; aborts and notifies user if exceeded.
3. Triggers resolution + payment exactly as in 4.2, tagged with the recurrence cycle number for audit/reconciliation.
4. Persists result (success/failure) and emits an event the wallet UI can surface to the user.

5. API Specification (Resolver Service)

Method & Path	Purpose	Notes
`POST /offers`	Register a new Offer	Body: encoded Offer + signature; verified server-side before storage
`GET /offers/:id`	Fetch raw Offer metadata	Public, cacheable
`POST /offers/:id/invoice`	Resolve an Offer into a fresh single-use invoice	Body: requested amount (if within bounds) and asset selection; calls recipient node RPC to mint
`GET /.well-known/fiberoffer/:user`	Fiber Address resolution	Mirrors LNURL-pay's well-known convention; returns the canonical Offer for that username
`DELETE /offers/:id`	Revoke an Offer	Requires signature from the original node key
`GET /offers/:id/recurrence-status`	Return cycles paid, cap remaining, next due date	Used by wallet UI to display standing-order state

6. Security & Trust Model

- **Signature verification:** every Offer must be signed by the recipient's Fiber node key; the resolver rejects unsigned or invalidly-signed Offers at registration time.
- **Resolver is not custodial:** the resolver never holds funds or private keys — it only mints invoices via authenticated RPC calls to the recipient's own node and relays them.
- **Replay protection:** each minted invoice carries its own unique payment hash from the underlying Fiber invoice mechanism, exactly as today; reusing the Offer never means reusing a payment hash.
- **Recurring payment safety:** `spending_cap_total` is mandatory for any wallet implementing auto-pay; the spec requires wallets to surface an explicit approval screen and a visible, one-tap revocation path.
- **Resolver availability risk:** if the MVP's HTTP resolver is down, Offers using it cannot be resolved. This is documented as a known MVP limitation, with the native peer-message transport (stretch goal) as the long-term mitigation, mirroring how BOLT12 removed LNURL's server dependency.
- **Node authentication for minting:** the resolver-to-node RPC call must be authenticated (reusing the existing wallet-signature/JWT pattern) so only the legitimate Offer owner's service can mint invoices on their behalf.

7. Storage Requirements

Store	Used for
PostgreSQL	Durable Offer records, recurrence schedules, payment/cycle history for reconciliation
Redis	BullMQ job queue for recurrence scheduling; short-lived resolution rate-limiting/session state

8. Non-Functional Requirements

Category	Requirement
Latency	Offer resolution (HTTP round trip + invoice mint) should complete in under 2 seconds on devnet under normal conditions
Availability	Resolver service should be independently horizontally scalable; no in-memory state that prevents running multiple instances
Compatibility	Generated invoices must be indistinguishable from normal Fiber invoices to any existing wallet's payment path
Extensibility	Schema versioned from day one; unknown optional fields must be safely ignorable by older clients
Observability	Structured logs for every resolution attempt (success/failure/reason) to support debugging during the demo and beyond

9. Testing Strategy

- **Unit tests** for schema encode/decode round-trips and signature verification (valid + tampered payloads).
- **Integration tests** against a live Fiber devnet node for the full create → resolve → pay cycle.
- **Scenario tests** for recurrence: cap enforcement, revocation mid-cycle, and missed-cycle recovery.
- **Manual QA script** for the hackathon demo itself, rehearsed before recording the video deliverable.

10. Deployment & Environment

Environment piece	Detail
Fiber node	CKB devnet via `offckb`, running `fnn` per Fiber Network docs
Resolver service	Containerized (Docker), deployed to a small VPS or PaaS with public HTTPS for the demo
Redis/Postgres	Managed instances or Dockerized alongside the resolver for the hackathon window
Demo apps	Next.js merchant demo deployed publicly; Expo payer app runnable via QR/dev build

End of Technical Requirements Document. See the companion Functional Requirements Document for granular, testable requirements and acceptance criteria.